HurryMind Market Analysis Feature — Full Technical Specification

> **Purpose:** This document contains everything needed to implement the Market Analysis feature for HurryMind. It is designed to be fed directly into AI coding tools (Cursor, Claude Code) for rapid implementation.

---

1. Architecture Overview

Data Flow

```
User clicks "Run Market Analysis" on Idea Card
        ↓
[API Route] /api/analysis/run  (POST)
        ↓
Step 1: Context Assembly (instant)
  → Gather idea data, signal conversations, scores from your DB
        ↓
Step 2: Parallel Data Enrichment (3-8 sec)
  → Google Trends API  ←──┐
  → Serper.dev Search  ←──┤  (Promise.all — parallel)
  → Your signal DB query ←┘
        ↓
Step 3: Sequential AI Chain (20-40 sec, streamed via SSE)
  → Gemini Call 1: Problem & Customer Analysis
  → Gemini Call 2: Market Sizing (uses Call 1 output + Trends data)
  → Gemini Call 3: Competitive Landscape (uses Call 1 output + search results)
  → Gemini Call 4: Unit Economics (uses Call 2 + Call 3 outputs)
  → Gemini Call 5: Risk & Go/No-Go (uses all prior outputs)
        ↓
Each call's JSON result streamed to frontend via SSE
        ↓
Frontend renders each section progressively
```

Tech Stack

- **Framework:** Next.js 14+ (App Router)
- **AI:** Google Gemini 2.0 Flash (via @google/generative-ai SDK)
- **Search enrichment:** Serper.dev API ($50/mo, 10K searches)
- **Trends:** Google Trends API (free, use `google-trends-api` npm package)
- **Streaming:** Server-Sent Events (SSE) via Next.js Route Handlers

- **Charts:** Recharts (already likely in your stack)
- **Database:** Whatever you're using now — just need to store analysis results

---

2. Database Schema (new tables)

```sql
-- Store completed market analyses
CREATE TABLE market_analyses (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  idea_id     UUID NOT NULL REFERENCES ideas(id),
  user_id     UUID NOT NULL REFERENCES users(id),
  status      TEXT DEFAULT 'pending', -- pending | generating | complete | failed

  -- Section results (each stored as JSONB)
  problem_analysis    JSONB,
  market_sizing       JSONB,
  competitive_landscape JSONB,
  unit_economics      JSONB,
  risk_assessment     JSONB,
  go_no_go            JSONB,

  -- Metadata
  weighted_score      FLOAT,
  recommendation      TEXT, -- STRONG_GO | CONDITIONAL_GO | NO_GO
  enrichment_data     JSONB, -- raw data from Google Trends, Serper, etc.

  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ DEFAULT now()
);

-- Track usage against plan limits
-- (Add to your existing usage tracking, or create new)
CREATE TABLE analysis_usage (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id),
  analysis_id UUID REFERENCES market_analyses(id),
  created_at  TIMESTAMPTZ DEFAULT now()
);
```

---

3. API Routes

3a. Trigger Analysis — `POST /api/analysis/run`

```typescript
// app/api/analysis/run/route.ts

import { NextRequest, NextResponse } from 'next/server';

export async function POST(req: NextRequest) {
  const { ideaId } = await req.json();
  const user = await getAuthUser(req); // your auth

  // 1. Check plan limits
  const usage = await getMonthlyAnalysisCount(user.id);
  const limit = getPlanLimit(user.plan); // free=2, pro=10, scale=25
  if (usage >= limit) {
    return NextResponse.json({ error: 'Monthly analysis limit reached' }, { status: 429 });
  }

  // 2. Create analysis record
  const analysis = await db.marketAnalyses.create({
    ideaId, userId: user.id, status: 'pending'
  });

  // 3. Return analysis ID — frontend will connect to SSE stream
  return NextResponse.json({ analysisId: analysis.id });
}
```

3b. Stream Analysis — `GET /api/analysis/[id]/stream`

This is the core route. It runs the full pipeline and streams results section-by-section.

```typescript
```

```
// app/api/analysis/[id]/stream/route.ts

import { NextRequest } from 'next/server';
import { GoogleGenerativeAI } from '@google/generative-ai';

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY!);

export async function GET(
  req: NextRequest,
  { params }: { params: { id: string } }
) {
  const analysisId = params.id;

  const encoder = new TextEncoder();
  const stream = new ReadableStream({
    async start(controller) {
      const send = (event: string, data: any) => {
        controller.enqueue(encoder.encode(`event: ${event}\ndata: ${JSON.stringify(data)}\n\n`));
      };

      try {
        // Update status
        await db.marketAnalyses.update(analysisId, { status: 'generating' });

        // ── STEP 1: Context Assembly ──
        send('status', { step: 'context', message: 'Gathering signal data...' });

        const idea = await db.ideas.findById(/* from analysis record */);
        const signals = await db.signals.findByIdeaId(idea.id);
        // signals = array of { text, source, sentiment, timestamp, url }

        const contextPacket = {
          title: idea.title,
          description: idea.description,
          category: idea.category,
          compositeScore: idea.score,
          signals: signals.slice(0, 50), // top 50 most relevant
          signalStats: {
            total: signals.length,
            bySource: groupBy(signals, 'source'),
            avgSentiment: avg(signals.map(s => s.sentiment)),
            dateRange: { from: min(signals.map(s => s.timestamp)), to: max(signals.map(s => s.timestamp)) }
          }
```

```
};

// ── STEP 2: Parallel Data Enrichment ──
send('status', { step: 'enrichment', message: 'Fetching market data...' });

const [trendsData, searchResults] = await Promise.all([
  fetchGoogleTrends(idea.title, idea.category),
  fetchSerperResults(idea.title, idea.category),
]);

// ── STEP 3: Sequential AI Chain ──

// CALL 1: Problem & Customer Analysis
send('status', { step: 'problem', message: 'Analyzing problem & customers...' });
const problemResult = await runGeminiCall(
  PROMPTS.problemAnalysis(contextPacket)
);
send('section', { id: 'problem', data: problemResult });
await db.marketAnalyses.update(analysisId, { problem_analysis: problemResult });

// CALL 2: Market Sizing
send('status', { step: 'market', message: 'Calculating market size...' });
const marketResult = await runGeminiCall(
  PROMPTS.marketSizing(contextPacket, problemResult, trendsData)
);
send('section', { id: 'market', data: marketResult });
await db.marketAnalyses.update(analysisId, { market_sizing: marketResult });

// CALL 3: Competitive Landscape
send('status', { step: 'competitive', message: 'Mapping competitors...' });
const competitiveResult = await runGeminiCall(
  PROMPTS.competitiveLandscape(contextPacket, problemResult, searchResults)
);
send('section', { id: 'competitive', data: competitiveResult });
await db.marketAnalyses.update(analysisId, { competitive_landscape: competitiveResult });

// CALL 4: Unit Economics
send('status', { step: 'economics', message: 'Modeling revenue scenarios...' });
const economicsResult = await runGeminiCall(
  PROMPTS.unitEconomics(contextPacket, marketResult, competitiveResult)
);
send('section', { id: 'economics', data: economicsResult });
await db.marketAnalyses.update(analysisId, { unit_economics: economicsResult });
```

```
      // CALL 5: Risk & Go/No-Go
      send('status', { step: 'gonogo', message: 'Generating recommendation...' });
      const goNoGoResult = await runGeminiCall(
        PROMPTS.riskAndGoNoGo(contextPacket, problemResult, marketResult, competitiveResult, economicsResult)
      );
      send('section', { id: 'gonogo', data: goNoGoResult });
      await db.marketAnalyses.update(analysisId, {
        risk_assessment: goNoGoResult.risks,
        go_no_go: goNoGoResult.recommendation,
        weighted_score: goNoGoResult.weightedScore,
        recommendation: goNoGoResult.verdict,
        status: 'complete'
      });

      send('complete', { analysisId });

    } catch (error) {
      send('error', { message: error.message });
      await db.marketAnalyses.update(analysisId, { status: 'failed' });
    } finally {
      controller.close();
    }
  }
});

return new Response(stream, {
  headers: {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache',
    'Connection': 'keep-alive',
  },
});
}

// ── Helper: Run Gemini with JSON mode ──
async function runGeminiCall(prompt: string): Promise<any> {
  const model = genAI.getGenerativeModel({
    model: 'gemini-2.0-flash',
    generationConfig: {
      responseMimeType: 'application/json',
      temperature: 0.3, // low temp for analytical accuracy
    },
  });
```

```
  const result = await model.generateContent(prompt);
  const text = result.response.text();
  return JSON.parse(text);
}
```

3c. Get Completed Analysis — `GET /api/analysis/[id]`

```typescript
// Simple CRUD endpoint to retrieve a saved analysis
export async function GET(req: NextRequest, { params }) {
  const analysis = await db.marketAnalyses.findById(params.id);
  if (!analysis) return NextResponse.json({ error: 'Not found' }, { status: 404 });
  return NextResponse.json(analysis);
}
```

---

4. Data Enrichment Functions

4a. Google Trends

```typescript
```

```typescript
// lib/enrichment/google-trends.ts
import googleTrends from 'google-trends-api';

export async function fetchGoogleTrends(title: string, category: string) {
  try {
    // Extract 2-3 core keywords from the idea title
    const keywords = extractKeywords(title); // e.g., "AI code review"

    const results = await googleTrends.interestOverTime({
      keyword: keywords.slice(0, 3),
      startTime: new Date(Date.now() - 12 * 30 * 24 * 60 * 60 * 1000), // 12 months ago
      geo: '', // worldwide
    });

    const parsed = JSON.parse(results);
    return {
      timelineData: parsed.default.timelineData.map((point: any) => ({
        date: point.formattedAxisTime,
        values: point.value,
      })),
      averages: parsed.default.averages,
    };
  } catch (error) {
    console.error('Google Trends error:', error);
    return null; // non-critical, analysis can proceed without it
  }
}
```

4b. Serper.dev Search

```typescript
```

```typescript
// lib/enrichment/serper.ts

const SERPER_API_KEY = process.env.SERPER_API_KEY;

export async function fetchSerperResults(title: string, category: string) {
  const queries = [
    `${title} SaaS competitors`,
    `${title} software pricing`,
    `${category} market size 2025 2026`,
    `${title} alternatives reviews`,
  ];

  const results = await Promise.all(
    queries.map(q => searchSerper(q))
  );

  return {
    competitors: results[0],
    pricing: results[1],
    marketData: results[2],
    reviews: results[3],
  };
}

async function searchSerper(query: string) {
  const res = await fetch('https://google.serper.dev/search', {
    method: 'POST',
    headers: {
      'X-API-KEY': SERPER_API_KEY!,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({ q: query, num: 10 }),
  });
  return res.json();
}
```

---

5. Gemini Prompts (The Most Important Part)

Each prompt is designed to return structured JSON. The system instruction forces JSON-only output.

5a. Call 1: Problem & Customer Analysis

```typescript
```

```
// lib/prompts/index.ts

export const PROMPTS = {
  problemAnalysis: (context: ContextPacket) => `
You are a senior product strategist conducting a market analysis for a potential SaaS product.

IDEA: ${context.title}
DESCRIPTION: ${context.description}
CATEGORY: ${context.category}

REAL CONVERSATIONS FROM POTENTIAL USERS (these are actual posts from Reddit, Hacker News, Twitter, and Pro

${context.signals.slice(0, 30).map((s, i) => `[${s.source}] "${s.text}" (sentiment: ${s.sentiment})`).join('\n')}

SIGNAL STATISTICS:
- Total conversations found: ${context.signalStats.total}
- Sources: ${JSON.stringify(context.signalStats.bySource)}
- Average sentiment: ${context.signalStats.avgSentiment}
- Date range: ${context.signalStats.dateRange.from} to ${context.signalStats.dateRange.to}

Based ONLY on evidence from these real conversations, produce the following analysis.

Respond with ONLY valid JSON matching this exact schema (no markdown, no backticks, no preamble):

{
  "jtbd": "When [situation from conversations], I want to [motivation], so I can [expected outcome]",
  "painPoints": [
    {
      "text": "Description of pain point derived from conversations",
      "frequency": <number of conversations mentioning this>,
      "sentiment": <float from -1.0 to 0, more negative = more painful>,
      "exampleQuotes": ["direct short quote 1", "direct short quote 2"]
    }
  ],
  "icp": {
    "who": "Primary persona description",
    "companySize": "Company size / team size range",
    "revenue": "Revenue range of target companies",
    "techStack": "Common technologies mentioned in conversations",
    "behavior": "How they currently solve this problem",
    "buyingTrigger": "What event would make them search for a solution"
  },
  "customerSegments": [
```

```
    {
      "name": "Segment name",
      "description": "Who they are",
      "estimatedPctOfMarket": <0-100>,
      "willingnessToPaySignal": "low | medium | high",
      "evidence": "Why this segment exists based on conversations"
    }
  ],
  "buyerVsUser": "Explanation of whether buyer and user are the same person",
  "confidence": "low | medium | high",
  "evidenceBasis": "Summary of evidence quality and any gaps"
}
`,

  // ────────────────────────────────────────────────────────────────────────

  marketSizing: (context: ContextPacket, problemResult: any, trendsData: any) => `
You are a senior market analyst calculating TAM/SAM/SOM for a potential SaaS product.

IDEA: ${context.title}
CATEGORY: ${context.category}

CUSTOMER ANALYSIS (from prior step):
${JSON.stringify(problemResult, null, 2)}

GOOGLE TRENDS DATA (search interest over time):
${trendsData ? JSON.stringify(trendsData.timelineData?.slice(-12)) : 'Not available'}

SIGNAL DATA:
- ${context.signalStats.total} real conversations found expressing this pain point
- Average sentiment: ${context.signalStats.avgSentiment}

INSTRUCTIONS:
1. Calculate TAM using BOTH top-down and bottom-up methods
2. Bottom-up is primary: count target organizations × realistic ACV
3. SAM = TAM filtered by geography, segment, and product capability
4. SOM = realistic 3-year capture (typically 1-5% of SAM for new entrants)
5. Cross-reference: if top-down and bottom-up differ by >30%, flag this

Respond with ONLY valid JSON:

{
  "tam": {
    "value": <number in USD>,
```

```
    "label": "$X.XB or $XXXM",
    "bottomUp": {
      "targetOrganizations": <count>,
      "acv": <annual contract value USD>,
      "calculation": "description of how you arrived at org count",
      "sources": ["data sources used"]
    },
    "topDown": {
      "totalMarket": <broader market USD>,
      "filterApplied": "description of filter",
      "result": <filtered number>
    },
    "alignmentCheck": "Do top-down and bottom-up align? By how much?"
  },
  "sam": {
    "value": <number>,
    "label": "$X.XB or $XXXM",
    "filtersApplied": ["geographic filter", "segment filter", "capability filter"],
    "methodology": "Explanation"
  },
  "som": {
    "value": <number>,
    "label": "$X.XM",
    "captureRate": <0.01-0.05>,
    "timeframe": "3 years",
    "assumptions": ["assumption 1", "assumption 2"],
    "targetCustomers": <number of paying customers needed>
  },
  "demandTrajectory": {
    "direction": "growing | stable | declining",
    "growthRate": "X% description",
    "trendsInsight": "What Google Trends shows",
    "signalTrend": "Are conversations increasing or decreasing over time?"
  },
  "assumptions": ["key assumption 1", "key assumption 2", "key assumption 3"],
  "risks": ["What could make this sizing wrong?"],
  "confidence": "low | medium | high"
}
`,

  // ────────────────────────────────────────────────────────────────────────

  competitiveLandscape: (context: ContextPacket, problemResult: any, searchResults: any) => `
You are a competitive intelligence analyst mapping the landscape for a potential SaaS product.
```

```
IDEA: ${context.title}
CATEGORY: ${context.category}
TARGET CUSTOMER: ${JSON.stringify(problemResult.icp)}

SEARCH RESULTS FOR COMPETITORS AND ALTERNATIVES:
${JSON.stringify(searchResults.competitors?.organic?.slice(0, 10))}

SEARCH RESULTS FOR PRICING:
${JSON.stringify(searchResults.pricing?.organic?.slice(0, 10))}

SEARCH RESULTS FOR REVIEWS:
${JSON.stringify(searchResults.reviews?.organic?.slice(0, 10))}

INSTRUCTIONS:
1. Identify 4-8 competitors (mix of direct, indirect, and "do nothing" alternatives)
2. For positioning map, choose the 2 dimensions most important to the TARGET CUSTOMER
3. Score each competitor 0-100 on both dimensions
4. Place "Your Product" where it would logically sit given the identified gaps
5. Extract real pricing data from search results where possible
6. Identify specific gaps and opportunities

Respond with ONLY valid JSON:

{
  "positioningMap": {
    "xAxis": { "label": "Dimension 1 name (e.g. Feature Depth)", "description": "What this means" },
    "yAxis": { "label": "Dimension 2 name (e.g. Solo-Dev Focus)", "description": "What this means" }
  },
  "competitors": [
    {
      "name": "Competitor name",
      "type": "direct | indirect | doNothing",
      "x": <0-100 position on X axis>,
      "y": <0-100 position on Y axis>,
      "funding": "Funding amount or 'Bootstrapped' or 'Unknown'",
      "pricing": "Pricing description",
      "pricingModel": "freemium | subscription | usage-based | hybrid",
      "strengths": ["strength 1", "strength 2"],
      "weaknesses": ["weakness 1", "weakness 2"],
      "targetAudience": "Who they primarily serve",
      "estimatedTraffic": "high | medium | low (based on search presence)"
    }
  ],
```

```
  "yourProduct": {
    "x": <suggested position>,
    "y": <suggested position>,
    "suggestedPositioning": "How to differentiate based on gap analysis"
  },
  "marketGaps": [
    {
      "gap": "Description of unserved need",
      "evidence": "Why this gap exists (from reviews, search data, conversations)",
      "opportunity": "How to exploit this gap"
    }
  ],
  "reviewIntelligence": {
    "topComplaints": [
      { "complaint": "Description", "frequency": "X% of negative reviews", "opportunity": "How you could solve this" }
    ],
    "unmetNeeds": ["Need 1 from reviews", "Need 2"]
  },
  "incumbentThreat": {
    "level": "low | medium | high",
    "biggestThreat": "Who and why",
    "likelyResponse": "How incumbents will react to your entry",
    "timeToResponse": "Estimated months before they copy your differentiator"
  },
  "confidence": "low | medium | high"
}
`,

  // ────────────────────────────────────────────────────────────────────────

  unitEconomics: (context: ContextPacket, marketResult: any, competitiveResult: any) => `
You are a SaaS financial analyst building revenue projections for a potential product.

IDEA: ${context.title}
MARKET SIZING: ${JSON.stringify(marketResult)}
COMPETITIVE PRICING: ${JSON.stringify(competitiveResult.competitors?.map((c: any) => ({ name: c.name, pricing: c.p
TARGET SOM: ${marketResult.som?.label} (${marketResult.som?.targetCustomers} customers)

INSTRUCTIONS:
1. Model THREE scenarios: Conservative, Moderate, Aggressive
2. For each: set ACV, monthly new user acquisition rate, churn rate, CAC
3. Project MRR and user count at 12 and 24 months
4. Calculate LTV, LTV:CAC ratio, CAC payback months
5. Compare against SaaS industry benchmarks (LTV:CAC 3:1, churn 3-5%/mo for SMB, gross margin 70-85%)
```

6. Suggest optimal pricing tier structure

Respond with ONLY valid JSON:

```
{
  "suggestedPricing": {
    "model": "freemium | subscription | usage-based | hybrid",
    "rationale": "Why this model fits",
    "tiers": [
      { "name": "Tier name", "price": <monthly USD>, "features": ["feature1", "feature2"], "targetSegment": "Who" }
    ]
  },
  "scenarios": [
    {
      "name": "Conservative",
      "acv": <annual contract value>,
      "monthlyPrice": <price>,
      "assumptions": {
        "newUsersPerMonth": <starting rate>,
        "userGrowthRate": <monthly growth %>,
        "monthlyChurnRate": <percent>,
        "cac": <cost to acquire one customer>,
        "grossMargin": <percent>
      },
      "month12": {
        "users": <total paying users>,
        "mrr": <monthly recurring revenue>,
        "arr": <annual recurring revenue>
      },
      "month24": {
        "users": <total>,
        "mrr": <mrr>,
        "arr": <arr>
      },
      "unitEconomics": {
        "ltv": <lifetime value>,
        "ltvCacRatio": <float>,
        "cacPaybackMonths": <integer>,
        "ruleOf40": <growth + margin percentage>
      }
    }
    // ... repeat for Moderate and Aggressive
  ],
  "benchmarkComparison": [
```

```
    {
      "metric": "Metric name",
      "yourModerate": "Your moderate scenario value",
      "industryBenchmark": "Industry standard",
      "status": "excellent | good | warning | danger",
      "interpretation": "What this means"
    }
  ],
  "breakEvenAnalysis": {
    "monthsToBreakEven": <integer per scenario>,
    "minimumViableUsers": <users needed to cover basic costs>,
    "assumptions": ["Fixed cost assumptions"]
  },
  "confidence": "low | medium | high"
}
`,

  // ────────────────────────────────────────────────────────────────

  riskAndGoNoGo: (
    context: ContextPacket,
    problemResult: any,
    marketResult: any,
    competitiveResult: any,
    economicsResult: any
  ) => `
You are a senior investment analyst making a go/no-go recommendation for building a SaaS product.

IDEA: ${context.title}
TOTAL SIGNALS: ${context.signalStats.total} real conversations expressing this pain point

ANALYSIS COMPLETED SO FAR:
- Problem Analysis: ${JSON.stringify(problemResult)}
- Market Sizing: TAM ${marketResult.tam?.label}, SAM ${marketResult.sam?.label}, SOM ${marketResult.som?.label}
- Competitive Landscape: ${competitiveResult.competitors?.length} competitors identified, incumbent threat: ${competitiveI
- Unit Economics (Moderate): LTV:CAC ${economicsResult.scenarios?.[1]?.unitEconomics?.ltvCacRatio}, 24mo ARR ${ec

SCORING FRAMEWORK:
Rate each dimension 1-5 (5 = best / lowest risk) with these weights:
- Market Risk (25%): Is demand validated? Is the market growing?
- Competitive Risk (20%): Can incumbents crush you? Is there defensible positioning?
- Execution Risk (20%): Can a small team build this? Is the GTM motion clear?
- AI Displacement Risk (15%): Will AI agents replace this tool? Score using:
  - System of Record strength (1-4): Does it house authoritative data?
```

  - Non-Software Complements (1-4): Are there capabilities beyond pure code?
  - User & Usage Profile (1-4): Are users high-value decision-makers?
- Financial Risk (10%): Are unit economics viable? Is there a path to profitability?
- Regulatory Risk (10%): Are there compliance barriers?

DECISION THRESHOLDS:
- Weighted score >= 80% → STRONG GO
- Weighted score 60-80% → CONDITIONAL GO (must list specific conditions)
- Weighted score < 60% → NO GO

IMPORTANT: Be rigorous and honest. Do NOT inflate scores. If this idea shouldn't be built, say so clearly.

Respond with ONLY valid JSON:

```
{
  "risks": {
    "dimensions": [
      {
        "name": "Market Risk",
        "score": <1-5>,
        "weight": 0.25,
        "rationale": "Detailed explanation citing evidence from prior analysis",
        "mitigations": ["How to reduce this risk"]
      }
      // ... all 6 dimensions
    ],
    "aiDisplacement": {
      "systemOfRecord": { "score": <1-4>, "rationale": "explanation" },
      "nonSoftwareComplements": { "score": <1-4>, "rationale": "explanation" },
      "userProfile": { "score": <1-4>, "rationale": "explanation" },
      "overallVerdict": "Summary of AI displacement risk and how to mitigate"
    },
    "killRisks": [
      {
        "scenario": "What could kill this opportunity",
        "probability": "low | medium | high",
        "mitigation": "How to hedge against this"
      }
    ]
  },
  "recommendation": {
    "weightedScore": <float 0-100>,
    "verdict": "STRONG_GO | CONDITIONAL_GO | NO_GO",
    "headline": "One-sentence summary of recommendation",
```

```
    "radarScores": [
      { "dimension": "Market Demand", "score": <1-10> },
      { "dimension": "Market Size", "score": <1-10> },
      { "dimension": "Competitive Position", "score": <1-10> },
      { "dimension": "Unit Economics", "score": <1-10> },
      { "dimension": "Execution Feasibility", "score": <1-10> },
      { "dimension": "Defensibility", "score": <1-10> }
    ],
    "strengths": ["Top 3 strengths with evidence"],
    "weaknesses": ["Top 3 weaknesses with evidence"],
    "conditions": ["If CONDITIONAL_GO: specific conditions that must be met"],
    "nextSteps": [
      {
        "action": "What to do next",
        "timeline": "When to do it",
        "purpose": "Why this matters"
      }
    ]
  },
  "confidence": "low | medium | high"
}
`
};
```

---

6. Frontend Implementation

6a. Analysis Page — `app/analysis/[id]/page.tsx`

```typescript
```

```
'use client';

import { useEffect, useState } from 'react';
import { useParams } from 'next/navigation';

type SectionId = 'problem' | 'market' | 'competitive' | 'economics' | 'gonogo';
type SectionStatus = 'pending' | 'generating' | 'complete' | 'error';

interface AnalysisState {
  sections: Record<SectionId, { status: SectionStatus; data: any }>;
  statusMessage: string;
  isComplete: boolean;
}

export default function AnalysisPage() {
  const { id } = useParams();
  const [state, setState] = useState<AnalysisState>({
    sections: {
      problem: { status: 'pending', data: null },
      market: { status: 'pending', data: null },
      competitive: { status: 'pending', data: null },
      economics: { status: 'pending', data: null },
      gonogo: { status: 'pending', data: null },
    },
    statusMessage: 'Initializing...',
    isComplete: false,
  });

  useEffect(() => {
    // First check if analysis already exists (page reload)
    fetch(`/api/analysis/${id}`)
      .then(res => res.json())
      .then(data => {
        if (data.status === 'complete') {
          // Load from DB, populate all sections
          setState(loadCompletedAnalysis(data));
          return;
        }
        // Otherwise, connect to SSE stream
        connectToStream();
      });

    function connectToStream() {
```

```
    const eventSource = new EventSource(`/api/analysis/${id}/stream`);

    eventSource.addEventListener('status', (e) => {
      const { step, message } = JSON.parse(e.data);
      setState(prev => ({
        ...prev,
        statusMessage: message,
        sections: {
          ...prev.sections,
          [step]: { ...prev.sections[step as SectionId], status: 'generating' },
        },
      }));
    });

    eventSource.addEventListener('section', (e) => {
      const { id: sectionId, data } = JSON.parse(e.data);
      setState(prev => ({
        ...prev,
        sections: {
          ...prev.sections,
          [sectionId as SectionId]: { status: 'complete', data },
        },
      }));
    });

    eventSource.addEventListener('complete', () => {
      setState(prev => ({ ...prev, isComplete: true, statusMessage: 'Analysis complete' }));
      eventSource.close();
    });

    eventSource.addEventListener('error', (e) => {
      console.error('SSE error:', e);
      eventSource.close();
    });

    return () => eventSource.close();
  }
}, [id]);

// Render: use the collapsible section pattern from the React prototype
// Each section checks its status and renders accordingly:
// - pending: greyed out, locked
// - generating: pulsing indicator
// - complete: fully rendered with data
```

```
  return (
    <div>
      {/* Use the React prototype component structure */}
      {/* Each section component receives state.sections[sectionId].data */}
    </div>
  );
}
```

6b. Trigger Button — Add to Idea Card

```typescript
// components/IdeaCard.tsx — add this button

function RunAnalysisButton({ ideaId }: { ideaId: string }) {
  const [loading, setLoading] = useState(false);
  const router = useRouter();

  const handleClick = async () => {
    setLoading(true);
    try {
      const res = await fetch('/api/analysis/run', {
        method: 'POST',
        body: JSON.stringify({ ideaId }),
      });
      const { analysisId } = await res.json();
      router.push(`/analysis/${analysisId}`);
    } catch (error) {
      // Handle plan limit error, etc.
    } finally {
      setLoading(false);
    }
  };

  return (
    <button onClick={handleClick} disabled={loading}>
      {loading ? 'Starting...' : '🔬 Run Market Analysis'}
    </button>
  );
}
```

7. Environment Variables

```env
# .env.local

# Gemini AI
GEMINI_API_KEY=your_gemini_api_key

# Serper.dev (sign up at serper.dev — $50/mo for 10K searches)
SERPER_API_KEY=your_serper_api_key

# Google Trends — no API key needed (uses google-trends-api npm package)
```

---

8. NPM Dependencies to Install

```bash
npm install @google/generative-ai google-trends-api recharts
```

---

9. Cost Per Analysis Run (Reference)

Component	Cost per Analysis	Notes
Gemini 2.0 Flash (5 calls)	~$0.02-0.05	~3K input + ~2K output tokens per call
Serper.dev (4 searches)	~$0.02	$50/mo = 10K searches
Google Trends	Free	npm package, no API key
Total	**~$0.04-0.07**	Excellent margin at $29/mo for 10 runs

---

10. Implementation Checklist (1-Day Sprint)

Morning (4 hours)

- [ ] Set up database tables (market_analyses, usage tracking)
- [ ] Create `/api/analysis/run` route (trigger + plan limit check)

- [ ] Create `/api/analysis/[id]/stream` route (SSE streaming)
- [ ] Implement `fetchGoogleTrends()` enrichment function
- [ ] Implement `fetchSerperResults()` enrichment function
- [ ] Implement `runGeminiCall()` with JSON mode
- [ ] Add all 5 prompts from Section 5

Afternoon (4 hours)

- [ ] Create analysis page `app/analysis/[id]/page.tsx` with SSE client
- [ ] Port React prototype components (from the .jsx file) into your app
- [ ] Wire up section data to components
- [ ] Add "Run Market Analysis" button to idea cards
- [ ] Add loading/generating states and animations
- [ ] Add plan limit enforcement and upgrade prompts
- [ ] Test end-to-end with a real idea

Evening (2 hours)

- [ ] Add "Export PDF" functionality (use @react-pdf/renderer or html2pdf.js)
- [ ] Add analysis history page (list past analyses)
- [ ] Polish animations and error states
- [ ] Deploy

---

11. JSON Output Schemas (Quick Reference)

For each Gemini call, here are the top-level keys you should expect and render:

Call	Top-Level Keys	Primary UI Component
Problem	jtbd, painPoints[], icp, customerSegments[], buyerVsUser	Pain point list with frequency bars
Market	tam, sam, som, demandTrajectory, assumptions[]	TAM/SAM/SOM funnel + area chart
Competitive	positioningMap, competitors[], yourProduct, marketGaps[], reviewIntelligence	Scatter plot + gap cards
Economics	suggestedPricing, scenarios[3], benchmarkComparison[]	Scenario tabs + bar chart + benchmark table
Go/No-Go	risks.dimensions[], risks.aiDisplacement, recommendation	Radar chart + score ring + conditions list

12. Error Handling Patterns

```typescript
// Wrap each Gemini call with retry logic
async function runGeminiCallWithRetry(prompt: string, maxRetries = 2): Promise<any> {
  for (let attempt = 0; attempt <= maxRetries; attempt++) {
    try {
      const result = await runGeminiCall(prompt);
      return result;
    } catch (error) {
      if (attempt === maxRetries) throw error;
      // Gemini rate limits: wait and retry
      await new Promise(r => setTimeout(r, 1000 * (attempt + 1)));
    }
  }
}

// If a section fails, still continue with remaining sections
// Send partial results and flag the failed section
send('section_error', { id: 'market', error: 'Failed to generate market sizing. Retry available.' });
```

---

13. Future Enhancements (Post-Launch)

1. **Crunchbase API** — Structured competitor data (funding, employee count, founding date)
2. **SimilarWeb API** — Competitor traffic estimates
3. **PDF Export** — Polished branded report for sharing
4. **Comparison Mode** — Compare analyses side-by-side for multiple ideas
5. **Custom Weights** — Let users adjust go/no-go scoring weights
6. **Proprietary Benchmark DB** — Aggregate anonymized data from all HurryMind analyses to build unique benchmarks
7. **Competitor Monitoring** — Alert when new competitors emerge (using your existing scraping infra)